

06

Directives & Pipes

© Gelişim · Junior → Mid-Level

📌 Bu modülde ne öğreneceğiz?

- ✓ Directive'in ne olduğunu ve üç türünü (component / attribute / structural)
- ✓ Dinamik sınıf ve stil bağlamayı (`[class.x]`, `[style.x]`, `[class]`, `[style]`)
- ✓ Kendi attribute directive'ini yazmayı (`host` metadata + signal'ler)
- ✓ Pipe'in ne olduğunu ve yerleşik pipe'ları (`date`, `currency`, `number`, `percent` ...)
- ✓ Kendi pipe'ını yazmayı (`@Pipe`, `transform`, pure/impure)
- ✓ Template reference variable (`#name`) ile elemanlara/örneklere şablondan erişmeyi

6.1 Directive Nedir?

6.2 Dinamik Sınıf & Stil

6.3 Custom Directive

6.4 Pipe & Yerleşikler

6.5

Custom Pipe

6.6 #name

Mini Proje

Özet

Ders 6.1

Directive Nedir?

Şimdiye kadar hep component yazdın. **Directive**, aslında zaten kullandığın ama adını koymadığın bir kavram — hatta component bile bir directive türüdür.

Nedir?

Directive, bir HTML elemanına ek davranış ya da görünüm kazandıran sınıftır. Elemanın kendisini oluşturmak zorunda değildir; var olan bir elemana “eklenir” ve onu zenginleştirir. Angular'da üç tür vardır:

Tür	Ne yapar	Örnek
Component	Şablonu olan directive (kendi görünümünü üretir)	<code><app-product-card></code>
Attribute directive	Var olan elemanın görünüm/davranışını değiştirir	<code>[appHighlight]</code> , <code>routerLink</code>
Structural directive	DOM'a eleman ekler/çıkarır (yapıyı değiştirir)	<code>@if</code> / <code>@for</code> (control flow)

❏ Structural işleri artık control flow yapıyor

Elemanı gösterip gizleme ya da liste üretme gibi “yapısal” işleri sen zaten `@if` / `@for` / `@switch` ile yapıyorsun (Modül 4). Bu yüzden bu modülde asıl odağımız **attribute directive'ler** ve onları kendin yazmak olacak.

Neden var? (Hangi derde deva?)

Aynı davranışı birçok elemanda tekrarlamamak için. Örneğin “üzerine gelince vurgula”, “otomatik odaklan”, “duruma göre renklendir” gibi bir davranışı bir kez directive olarak yazıp, ihtiyaç duyan her elemana tek bir öznitelikle eklersin. Tekrar yok, tek yerden bakım var.

Nasıl yazıldığını 6.3'te derinlemesine göreceğiz; önce dinamik sınıf/stil ile ısınalım.

✓ Bu Dersten Çıkarılacaklar

- ✓ Directive = bir elemana davranış/görünüm ekleyen sınıf.
- ✓ Üç tür: component (şablonlu), attribute (davranış/görünüm), structural (yapı).
- ✓ Yapısal işler artık `@if` / `@for` ile yapılır; odağımız attribute directive'ler.
- ✓ Amaç: tekrar eden davranışı tek yerde toplayıp birçok elemana uygulamak.

Ders 6.2

✨ Dinamik Sınıf & Stil

En sık ihtiyaç duyacağın “görünümü duruma göre değiştirme” işini binding'lerle yaparsın. Tek sınıf/stili zaten görmüştün; şimdi hepsini bir araya getirip birden çok sınıf/stili de ele alalım.

Tek Sınıf / Tek Stil

HTML

```
<!-- Koşul true ise 'active' sınıfı eklenir -->
<li [class.active]="isSelected()">...</li>

<!-- Tek bir stil özelliği -->
<span [style.color]="isError() ? 'red' : 'green'">...</span>

<!-- Birim gerektiren stiller: .px, .% ekle -->
<div [style.width.%]="progress()"></div>
```

⚠ Sık hata: `[class]="done"` ile tek sınıf açmak

Tek bir sınıfı koşula bağlamak istiyorsan `[class.done]="isDone()"` yazarsın. `[class]="done"` ise **tüm sınıf listesini** `done` değişkeniyle değiştirmeye çalışır — amaçladığın şey bu değildir. Nokta ile sınıf adını belirtmeyi unutma: `[class.SINIF]`.

Birden Çok Sınıf: `[class]` nesnesi

Aynı anda birkaç sınıfı koşullara bağlamak istersen `[class]` 'a bir **nesne** verirsin: anahtar = sınıf adı, değer = koşul.

HTML

```
<li [class]="{
  active: isSelected(),
  done: isDone(),
  urgent: priority() === 'high'
}">...</li>
```

Koşulu `true` olan sınıflar eklenir, `false` olanlar çıkar. Dizi de verebilirsin: `[class]="['card', theme()]"`.

Birden Çok Stil: `[style]` nesnesi

HTML

```
<div [style]="{
  color: textColor(),
  'font-weight': isBold() ? '700' : '400',
  'width.%': progress()
}">...</div>
```

ngClass / ngStyle — ne zaman?

Aynı işi yapan `ngClass` ve `ngStyle` directive'leri de vardır. Modern Angular'da yukarıdaki **yerleşik** `[class]` / `[style]` bağlamaları tercih edilir (ekstra import gerektirmez, daha doğrudan). `ngClass` 'i yalnızca çok dinamik, karmaşık sınıf mantığında ve bir sebebin varsa kullan; çoğu durumda gerek kalmaz.

🔗 Mini Quiz

1. Tek bir sınıfı koşula bağlamak için ne yazarsın?

↳ `[class.aktifSinif]="kosul()"` .

2. `[class]="done"` neden çoğu zaman istediğin şey değildir?

↳ Tek sınıf eklemeyi; tüm sınıf listesini `done` değeriyle değiştirmeye çalışır.

3. Aynı anda üç sınıfı koşullara bağlamak istersen?

↳ `[class]` 'a bir nesne ver: `{ a: kosulA(), b: kosulB(), c: kosulC() }` .

✗ Sık Yapılan Hatalar

✗ `[class]="done"` ile tek sınıf açmaya çalışmak. Tek sınıf için `[class.done]` kullan.

✗ Birim unutmak: `[style.width]="50"` çalışmaz; `[style.width.]="50"` ya da `[style.width.px]` .

✗ Gereksiz `ngClass` : basit koşullarda yerleşik `[class.x]` / `[class]` daha temiz.

✓ Bu Dersten Çıkarılacaklar

✓ Tek sınıf: `[class.x]="cond"` ; tek stil: `[style.prop]="val"` (gerekliyorsa `.px` / `.%`).

✓ Birden çok sınıf: `[class]="{ a: condA(), b: condB() }"` (nesne) veya dizi.

✓ Birden çok stil: `[style]="{ color: c(), 'width.%': p() }"` .

✓ Modern Angular'da yerleşik `[class]` / `[style]` tercih edilir; `ngClass` / `ngStyle` istisna.

Ders 6.3

<> Kendi Directive'ini Yazmak

İşin en güçlü kısmı: kendi attribute directive'ini yaz, bir davranışı bir kez tanımla, istediğin her elemana tek öznitelikle uygula. Örneğimiz klasik: **üzerine gelince vurgulama**.

iskelet: `@Directive` + `host`

Directive'in **şablonu yoktur**; eklendiği elemanı (host) doğrudan etkiler. Host elemanına bağlanmanın yolu, decorator içindeki `host` **nesnesidir**: hem stil/sınıf bağlarsın hem olay dinlersin.

TYPESCRIPT

```
import { Directive, input, signal, computed } from '@angular/core';

@Directive({
  selector: '[appHighlight]', // <div appHighlight> diye eklenir
  host: {
    '[style.backgroundColor]': 'background()', // stil bağlama
    '(mouseenter)': 'onEnter()', // olay dinleme
    '(mouseleave)': 'onLeave()',
  },
})
export class HighlightDirective {
  // Dışarıdan ayar: <div appHighlight [color]='gold'>
  readonly color = input('yellow');

  private readonly isHovered = signal(false);

  // Host binding'ler signal'lere tepki verir (tıpkı şablon gibi)
  protected readonly background = computed(() =>
    this.isHovered() ? this.color() : 'transparent',
  );

  protected onEnter(): void { this.isHovered.set(true); }
  protected onLeave(): void { this.isHovered.set(false); }
}
```

Kilit fikir: directive'in şablonu olmadığı için signal'leri **host binding'lerde** kullanırsın. `isHovered` değişince `background` yeniden hesaplanır, host elemanının arka planı kendiliğinden güncellenir — tıpkı bir component şablonundaki gibi.

Kullanımı

TYPESCRIPT

```
// Kullanan component'in imports'ına ekle
imports: [HighlightDirective],
```

HTML

```
<p appHighlight>Üzerime gel, vurgulanayım.</p>
<p appHighlight [color]='#fde68a'>Rengi de verebilirim.</p>
```

selector'daki [] ile binding'deki [] aynı şey değil

Kafa karıştıran bir nokta: `selector: '[appHighlight]'` içindeki köşeli parantez bir **CSS öznitelik seçicisidir** (“bu öznitelik bulunan elemanlar”); şablondaki `[color]="..."` ise **property binding**'dir (“bu input'a şu ifadenin değerini bağla”). Aynı karakter, iki farklı iş.

Bu yüzden directive'i **uygulamak** için parantez gerekmez; değer geçirmiyorsan düz öznitelik yazarsın. Parantez yalnızca bir input'a değer bağlarken gerekir:

Yazım	Anlamı
<code><p appHighlight></code>	Directive uygulanır, değer geçmez
<code><p appHighlight="gold"></code>	Aynı adlı input'a sabit metin (“gold”)
<code><p [appHighlight]="expr"></code>	Aynı adlı input'a ifadenin değeri bağlanır
<code><p [appHighlight]></code>	Geçersiz — binding'in bir değeri olmalı

Tutarlılık kalıbı: asıl input'u selector adıyla adlandır

Yerleşiklerin (`ngClass`, `routerLink`, `ngSrc`) hep parantezli görünmesinin sebebi, hepsinin selector'ıyla **aynı adda bir asıl input'u** olmasıdır. Kendi directive'inde de “tek bir asıl değer” varsa aynı kalıbı kullan: input'u selector adıyla adlandır — böylece tek yazımla hem uygulanır hem değer geçer.

```
TYPESCRIPT

@Directive({
  selector: '[appHighlight]',
  host: { '[style.backgroundColor]': 'appHighlight()' },
})
export class HighlightDirective {
  // selector ile AYNI ad -> [appHighlight]="gold" tek yazımı çalışır
  readonly appHighlight = input('yellow');
}
```

İç kodda `this.appHighlight()` okumak anlamı gizleyebilir. Çözüm: dışarıda selector adı, içeride anlamlı ad — `alias` ile:

```
TYPESCRIPT

readonly color = input('yellow', { alias: 'appHighlight' });
// şablonda: [appHighlight]="gold" | sınıfta: this.color()
```

❏ Her directive'e uygun değil

Bu kalıp yalnızca directive'in **tek bir asıl değeri** olduğunda uygundur. Birden çok ayar varsa asıl değer selector adıyla, ikincil ayarlar ayrı input'larla gider (`routerLink` 'in `queryParams` 'ı gibi). Hiç değer almayan bir directive'de (`appAutofocus` gibi) zaten input olmaz.

Host elemanına erişmek: `inject(ElementRef)`

Bazen host elemanının kendisine (odaklanma, ölçüm) ihtiyaç duyarsın. `ElementRef` 'i enjekte edersin:

TYPESCRIPT

```
import { Directive, ElementRef, inject, afterNextRender } from '@angular/core';

@Directive({ selector: '[appAutofocus]' })
export class AutofocusDirective {
  private readonly host = inject(ElementRef<HTMLElement>);

  constructor() {
    afterNextRender(() => this.host.nativeElement.focus());
  }
}
```

🔗 `host` metadata'yı tercih et

Host'a bağlanmanın bir de decorator'lı yolu vardır (`@HostBinding` / `@HostListener`), ama Angular bunları yalnızca geriye dönük uyumluluk için tutuyor. **Yeni kodda `host` nesnesini kullan** — daha derli toplu ve önerilen yol budur.

🔗 Mini Quiz

1. Directive'in şablonu olmadığına göre signal'leri nerede kullanırsın?
↳ Host binding'lerde (`host` nesnesi); değişince host elemanı güncellenir.
2. Host elemanına bir olay dinleyicisi nasıl bağlarsın?
↳ `host` nesnesinde `'(event)': 'method()'` ile.
3. Host'a bağlanırken `@HostBinding` mi `host` metadata mı?
↳ `host` metadata (decorator'lar sadece geriye uyumluluk için).

✗ Sık Yapılan Hatalar

- ✗ Directive'e şablon koymaya çalışmak. Şablon component'in işi; directive host elemanı etkiler.
- ✗ Selector'ı çok genel yapmak (örn. `div`). Dar, öznelik seçici kullan: `[appHighlight]` .
- ✗ `@HostBinding` / `@HostListener` ile başlamak. Yeni kodda `host` metadata.

✓ Bu Dersten Çıkarılacaklar

- ✓ `@Directive({ selector: '[appX]', host: {...} })` ile yazılır; şablonu yoktur.
- ✓ `host` nesnesinde stil/sınıf bağlar (`[style.x]`) ve olay dinlersin (`(event)`).
- ✓ Signal'ler host binding'lerde çalışır; `input()` ile dışarıdan ayarlanır.
- ✓ Host elemanına `inject(ElementRef)` ile erişilir. `host` metadata'yı tercih et.

Ders 6.4

✂ Pipe Nedir? & Yerleşik Pipe'lar

Modül 5 mini projesinde `number` pipe'ını kullanıp “import etmeyi” öğrenmiştin. Şimdi pipe'ların ne olduğunu ve en çok kullanacağın yerleşikleri tam görelim.

Nedir?

Pipe, bir değeri şablonda GÖSTERİRKEN dönüştüren küçük bir araçtır. Veriyi değiştirmez; yalnızca ekrandaki **görünümünü** biçimlendirir. Sözdizimi boru işaretiyledir: `{{ value | pipeName }}`.

Neden var?

Aynı biçimlendirme mantığını (tarih formatı, para birimi, büyük harf) her yerde elle yazmamak için. “1712 lira 5 kuruş”, “14.07.2026” gibi gösterimleri pipe tek satırda halleder; asıl veri (sayı, tarih nesnesi) olduğu gibi kalır.

Sık Kullanılan Yerleşik Pipe'lar

Pipe	Ne yapar	Örnek
<code>uppercase</code> / <code>lowercase</code>	Büyük / küçük harf	<code>{{ 'merhaba' uppercase }}</code> → MERHABA
<code>titlecase</code>	Her Kelimenin İlk Harfi	<code>{{ 'ürün adı' titlecase }}</code> → Ürün Adı
<code>number</code>	Sayı biçimi (ondalık)	<code>{{ 9.5 number:'1.2-2' }}</code> → 9.50
<code>currency</code>	Para birimi	<code>{{ 1712 currency:'TRY' }}</code> → ₺1,712.00
<code>percent</code>	Yüzde	<code>{{ 0.095 percent }}</code> → 10%
<code>date</code>	Tarih biçimi	<code>{{ order.date date:'dd/MM/yyyy' }}</code>
<code>slice</code>	Diziyi/metni kırp	<code>{{ items slice:0:3 }}</code>
<code>json</code>	Nesneyi JSON olarak göster (hata ayıklama)	<code>{{ product json }}</code>

Parametre ve Zincirleme

Pipe'a iki nokta ile parametre verirsın ve pipe'ları **zincirleyebilirsin** (soldan sağa uygulanır):

HTML

```
{{ order.date | date:'longDate' }}  
{{ price | currency:'TRY':'symbol':'1.2-2' }}  
{{ name | titlecase }}  
{{ product.title | slice:0:20 | uppercase }}
```

⚠ Pipe'ı `imports` 'a eklemeyi unutma

Yerleşik pipe'lar `@angular/common` 'dan gelir ve kullandığın component'in `imports` 'ına eklenmelidir (örn. `DatePipe`, `CurrencyPipe`). Eklemezsen `No pipe found with name ...` hatası alırsın — Modül 5'te `number` ile yaşadığın buydu.

📄 Pipe'lar “saftır” (pure)

Yerleşik pipe'lar varsayılan olarak **pure**'dur: yalnızca girdileri değiştiğinde yeniden çalışır, boşuna değil. Bu yüzden hızlıdır. Bu davranışı 6.5'te kendi pipe'ında da göreceğiz.

🔗 Mini Quiz

1. Pipe veriyi değiştirir mi?
↳ Hayır; yalnızca gösterimini (görünümünü) dönüştürür, asıl veri değişmez.
2. Bir sayıyı iki ondalıkla göstermek için hangi pipe + parametre?
↳ `number` pipe'ı, `| number:'1.2-2'`.
3. Pipe kullanınca `No pipe found` hatası neden çıkar?
↳ Pipe'ı component'in `imports` 'ına eklemediğin için.

✓ Bu Dersten Çıkarılacaklar

- ✓ Pipe, değeri gösterirken biçimlendirir: `{{ value | pipe:arg }}`; veriyi değiştirmez.
- ✓ Yerleşikler: `uppercase` / `titlecase` / `number` / `currency` / `percent` / `date` / `slice` / `json`.
- ✓ Parametre iki nokta ile verilir; pipe'lar zincirlenebilir.
- ✓ Kullanmadan önce pipe'ı `imports` 'a ekle; pipe'lar pure (verimli) çalışır.

Ders 6.5

🚀 Kendi Pipe'ını Yazmak

Yerleşikler yetmezse kendi pipe'ını yazarsın. Klasik örnek: uzun bir metni kısaltıp sonuna “...” ekleyen bir `truncate` pipe'ı.

İskelet: `@Pipe` + `transform`

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'truncate' })
export class TruncatePipe implements PipeTransform {
  // 1. parametre = pipe'a giren değer; sonrakiler = ':' ile verilenler
  transform(value: string, limit = 40, trail = '...'): string {
    if (!value) return '';
    return value.length > limit ? value.slice(0, limit) + trail : value;
  }
}
```

Kural basit: `transform` 'un **ilk parametresi** boru'nun soluna yazdığın değerdir; sonraki parametreler iki nokta ile verdiğin argümanlardır.

Kullanımı

```
imports: [TruncatePipe],
```

```
{{ product.description | truncate }}
{{ product.description | truncate:60 }}
{{ product.description | truncate:60:' →' }}
```

Pure vs Impure

Pipe'lar varsayılan olarak **pure**'dur: girdi **referansı** değişmedikçe `transform` yeniden çalışmaz. Bu, performans için harikadır ama bir tuzağı vardır: bir diziyi `push` ile içeriden değiştirirsen (referans aynı kalır) pure pipe bunu fark etmez.

```
@Pipe({ name: 'myFilter', pure: false }) // her değişim denetiminde çalışır
export class MyFilterPipe implements PipeTransform { /* ... */ }
```

⚠ Impure pipe'a çok güvenme

`pure: false` pipe her değişiklik denetiminde çalışır — sık ve maliyetli olabilir. Çoğu zaman doğru çözüm impure pipe değil, veriyi **immutable** güncellemek (yeni referans vermek) ya da işi bir `computed` 'e taşımaktır. Modül 5'teki immutability kuralını hatırla.

🔗 Mini Quiz

1. `transform` metodunun ilk parametresi nedir?
↳ Boru'nun soluna yazdığın değer (dönüştürülecek asıl değer).
2. Pipe'a `| truncate:60` dersin 60 nereye gider?
↳ `transform` 'un ikinci parametresine (`limit`).
3. Pure pipe bir diziyi `push` ile değiştirdiğinde neden güncellenmez?
↳ Pure pipe girdi referansına bakar; `push` referansı değiştirmez, bu yüzden fark edilmez.

✗ Sık Yapılan Hatalar

- ✗ Ağır iş için impure pipe kullanmak. Genelde `computed` ya da immutable güncelleme daha doğru.
- ✗ Pipe'ı `imports` 'a eklememek. `No pipe found` hatası verir.
- ✗ Pipe içinde yan etki yapmak (log, HTTP...). Pipe saf bir dönüştürücü olmalı.

✓ Bu Dersten Çıkarılacaklar

- ✓ `@Pipe({ name: 'x' }) + transform(value, ...args)` ile yazılır.
- ✓ `transform` 'un ilk parametresi giren değer; sonrakiler `:` ile verilen argümanlar.
- ✓ Pipe'lar pure'dur (girdi referansı değişmedikçe çalışmaz) → hızlı.
- ✓ `pure: false` nadiren gerekir; genelde immutable güncelleme / `computed` daha iyidir.

Ders 6.6

📄 Template Reference Variables (#name)

Şablonda bir elemana ya da bir directive/component örneğine isim verip aynı şablonda ona ulaşmanı sağlayan küçük ama çok kullanışlı bir araç. 6.3'te yazdığın directive'lere ve 5.5'teki `ViewChild` 'a doğrudan bağlanır.

Nedir?

`#name` , şablonundaki bir şeye (HTML elemanı, component ya da directive örneği) isim verip aynı şablonun başka yerinde ona erişmeni sağlar. Bir elemana etiket yapıştırıp sonra o etiketle “şuna bak” demek gibi.

Neden var?

Bir elemanın değeri ya da bir child'ın metoduna **TypeScript'e hiç gitmeden**, doğrudan şablonda erişmek için. En klasik örnek arama kutusu:

```
HTML

<input #box />
<button (click)="search(box.value)">Ara</button>
```

`#box` o `<input>` elemanının kendisidir (bir `HTMLInputElement`), dolayısıyla `box.value` yazılan metni verir — tek anlık okuma için pratik.

Üç şeyi işaret edebilir

1) **HTML elemanı** → o elemanın DOM nesnesi:

```
HTML

<input #box />          <!-- box: HTMLInputElement -->
<video #player></video> <!-- player.play(), player.pause() -->
```

2) **Component** → o component'in örneği (public alan/metotlarına erişirsin):

```
HTML

<app-rating #r></app-rating>
<button (click)="r.reset()">Sıfırla</button>
```

3) **Directive** (`exportAs` ile) → directive örneği:

```
HTML

<input [(ngModel)]="name" #ctrl="ngModel" />
@if (ctrl.invalid) { <span>Geçersiz</span> }
```

`#name` VS `@let` VS `viewChild`

Araç	Neyi isimlendirir	Nereden erişilir
<code>#name</code>	Bir elemanı/örneği	Aynı şablon içinde
<code>@let x = ...</code>	Bir ifadenin değerini	Aynı şablon içinde
<code>viewChild('name')</code>	<code>#name</code> 'li şeye	TypeScript sınıfından (signal)

Yani `#box` ile şablonda erişirsin; aynı `#box` 'a TS sınıfından erişmek istersen `viewChild('box')` kullanırsın (5.5). İkisi el ele çalışır.

⚠ Kapsam ve reaktiflik

`#name` yalnızca aynı şablonda geçerlidir; `@if` / `@for` blokları kendi kapsamını oluşturur (blok içindeki `#x` 'e dışarıdan erişemezsin). Ayrıca `box.value` anlık bir okumadır, signal gibi reaktif değildir; yazdıkça takip için `signal + (input)` kullan.

🔗 Mini Quiz

1. `#box` bir `<input>` 'a verildiğinde `box` ne olur?
↳ `box`, o input elemanının kendisi (`HTMLInputElement`) olur; `box.value` okunur.
2. `#name` ile `@let` farkı nedir?
↳ `#name` bir elemanı/örneği; `@let` bir ifadenin değerini isimlendirir.
3. `#box` 'a TypeScript sınıfından nasıl erişirsin?
↳ `viewChild('box')` ile (signal olarak).

✗ Sık Yapılan Hatalar

- ✗ `this.box` diye TS'te aramak. Şablon referansı TS'te otomatik yoktur; `viewChild('box')` kullan.
- ✗ `@if` / `@for` bloğundaki `#x` 'e dışarıdan erişmek. Blok kapsamı ayrıdır.
- ✗ `box.value` 'yu reaktif sanmak. Anlık okumadır; yazdıkça takip için `signal + (input)` .

✓ Bu Dersten Çıkarılacaklar

- ✓ `#name` = şablonda bir elemana/örneğe isim verip aynı şablonda erişme.
- ✓ Eleman → DOM nesnesi; component → örnek; directive → `exportAs` ile örnek.
- ✓ `@let` değeri, `#name` elemanı/örneği isimlendirir; TS erişimi `viewChild` ile.
- ✓ Kapsam şablon/blok ile sınırlı; `box.value` anlıktır, reaktif değildir.

Bölüm

🚩 Mini Proje: Sipariş Özeti — Pipe'lar & Directive

Bu modülün her şeyini tek ekranda birleştir: bir **sipariş listesi**. Her satır yerleşik pipe'larla biçimlenir; durum rozeti **kendi directive'inle** renklenir; uzun not **kendi pipe'ınla** kısaltılır.

Siparişler	
Pipe ve directive alıştırması	
Ayşe Yılmaz ₺1,712.00 %10 indirim Kapıda ödeme, akşam teslim tercih edildi; zili çalmadan...	14 Tem 2026 Kargoda
Mehmet Demir ₺429.90 %0 indirim Fatura adresi ile teslim adresi farklı, dikkat...	13 Tem 2026 Bekliyor
Zeynep Kaya ₺3,058.50 %15 indirim Hediye paketi istendi, not kartı eklendi.	11 Tem 2026 Teslim Edildi
Can Öztürk ₺89.00 %0 indirim Müşteri talebiyle iptal edildi.	9 Tem 2026 İptal

Sipariş listesi: tarih/tutar/indirim pipe'larla, durum rozeti directive ile renklenir

Özellikler / Gereksinimler

- Müşteri adı:** `titlecase` pipe ile düzgün yazım.
- Tarih:** `date` pipe ile biçimli (ör. `dd MMM yyyy`).
- Tutar:** `currency` pipe ile para birimi.
- İndirim:** `percent` pipe ile yüzde.
- Durum rozeti:** kendi directive'in (`[appStatusColor]`) duruma göre arka plan rengi versin (bekliyor → turuncu, kargoda → mavi, teslim → yeşil, iptal → kırmızı).
- Not:** kendi pipe'ın (`truncate`) uzun notu kısaltсын.

Angular tarafı (sende) — ipuçları

- Veri:** birkaç örnek siparişi bir `signal` 'de tut (id, customer, date, total, discount, status, note).
- Pipe import:** kullandığın yerleşikleri (`TitleCasePipe` , `DatePipe` , `CurrencyPipe` , `PercentPipe`) `imports` 'a ekle.
- Custom directive:** `[appStatusColor]="order.status"` → `input` ile durumu al, `host` ile `[style.backgroundColor]` bağla (`computed` ile renk seç).
- Custom pipe:** `truncate` pipe'ı yaz, nota uygula: `{{ order.note | truncate:50 }}`.
- Bonus:** üzerine gelince satırı vurgulayan bir `[appHighlight]` directive ekle.

📌 Materyaller

Statik referansı ayrıca verdim (JS yok): **index.html** (biçimlenmiş görünüm + renkli rozetler), **styles.scss**, **styles.css** ve ekran görüntüsü. HTML'de değerler zaten biçimli yazılıdır; senin işin bu görünümü **pipe'lar ve directive ile** üretmek. Bitince göster, birlikte gözden geçirelim.

✓ Bu Dersten Çıkarılacaklar

- ✓ Yerleşik pipe'lar (date/currency/percent/titlecase) gerçek bir tabloyu biçimlendirir.
- ✓ Custom directive (`host` + `input` + `computed`) ile duruma göre renk.
- ✓ Custom pipe (`truncate`) ile uzun metni kısaltma.
- ✓ Pipe'ları `imports` 'a eklemeyi unutma (Modül 5 refleksi).

Bölüm

🚩 Modül 6 Özeti

Artık şablonlarını hem davranış hem görünüm olarak zenginleştirebilirsin.

- **Directive türleri:** component / attribute / structural (yapısal işler `@if` / `@for` ile).
- **Dinamik sınıf/stil:** `[class.x]` , `[style.x]` , nesne ile `[class]` / `[style]` .
- **Custom directive:** `@Directive` + `host` metadata + signal'ler.
- **Pipe'lar:** gösterimde biçimlendirme; yerleşikler + `imports` 'a ekleme.
- **Custom pipe:** `@Pipe` + `transform` ; pure/impure.

📌 Sırada ne var?

Modül 7 — Services & Dependency Injection: veriyi ve mantığı component'lerden ayırıp paylaşılabılır servislere taşımak; `@Service` , `inject` ve DI'nın derinlemesine işlenişi.

Takıldığın her yeri sor — mini projeyi yapıp gösterdiğinde birlikte gözden geçiririz. Kolay gelsin!